

Learning from an Incompetent Teacher

IEEE Publication Technology, *Staff*, *IEEE*,

Abstract—end2end

Index Terms—Article submission, IEEE, IEEEtran, journal, LATEX, paper, template, typesetting.

I. INTRODUCTION

uav racing xxx 1)traditional perception-decision-planning, latency and error progration MPC-based, severely depend on accurate model

e2e vision planning has emerged as xxxxx, odometry-dependent global state estimation, odometry-freed, lack temporal all of the above, static gate no one dynamic gate, 1) perception model 2) spacialtemporal

in this work, we odom-free, end2end model to cross dymaic gates, contributiion: firstly propose visual-to-PWM action framework; MPPI in contrast to, sequenal image as input, predict the relative spacialtemporal, and map them in to the PWM-action

to enhance interprebility, multi-task supervision is added

II. METHODOLOGY

The agile flight task is formulated as a partially observable Markov decision process (POMDP) where the objective is to navigate through a sequence of dynamic gates while minimizing traversal time. The drone perceives the environment solely through a monocular RGB camera, and the learned policy network utilizes egocentric visual observations $\mathbf{o}_t$ to output low-level control commands $\mathbf{a}_t = [c, \omega_x, \omega_y, \omega_z]^\top$, where c represents collective thrust and ω denotes body rates.

Our approach integrates imitation learning (IL) and reinforcement learning (RL) through a unified training framework with dynamic loss weighting. Unlike conventional two-stage approaches that pre-train with IL before fine-tuning with RL, we simultaneously collect expert demonstrations and policy rollouts, dynamically adjusting the balance between IL and RL objectives throughout training. This enables the policy to leverage the sample efficiency of expert guidance while gradually gaining the ability to surpass expert performance through environment interactions.

A. Hybrid Data Collection

We maintain a hybrid data collection strategy that interleaves expert demonstrations with policy rollouts. Every f episodes (where f is the expert frequency), the drone is controlled by a Cross-Entropy Method based Model Predictive Controller (CEM-MPC) serving as the expert policy π_{expert} . In remaining episodes, the neural network policy π_θ is executed, enabling the collection of on-policy experience.

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

1) *Expert Policy (CEM-MPC)*: The MPC policy processes full state observations $\mathbf{s}_t = [\mathbf{p}^\top, \mathbf{v}^\top, \mathbf{R}^\top, \boldsymbol{\omega}^\top, \mathbf{d}^\top]^\top$, where $\mathbf{p}, \mathbf{v} \in \mathbb{R}^3$ denote position and velocity, $\mathbf{R} \in SO(3)$ denotes orientation, $\boldsymbol{\omega} \in \mathbb{R}^3$ denotes angular velocity, and $\mathbf{d} \in \mathbb{R}^3$ represents the relative position to the next gate. The CEM-MPC solves the following optimization at each timestep:

$$\mathbf{a}_t^* = \arg \min_{\mathbf{a}_{t:t+N}} \sum_{k=0}^{N-1} (\mathbf{s}_{t+k}^\top \mathbf{Q} \mathbf{s}_{t+k} + \mathbf{a}_{t+k}^\top \mathbf{R} \mathbf{a}_{t+k}) + \mathbf{s}_{t+N}^\top \mathbf{Q}_f \mathbf{s}_{t+N}, \quad (1)$$

where $\mathbf{Q}$, $\mathbf{R}$, and $\mathbf{Q}_f$ are state cost, control cost, and terminal cost matrices, respectively, and N is the prediction horizon.

2) *Hybrid Replay Buffer*: We maintain two replay buffers with distinct purposes:

- **Expert buffer** $\mathcal{B}_{\text{expert}}$: Stores transitions collected during MPC episodes, where the executed action is $\mathbf{a}_{\text{MPC}}$.
- **Dagger buffer** $\mathcal{B}_{\text{dagger}}$: Stores transitions from both MPC and NN policy episodes, where both the NN action $\mathbf{a}_{\text{NN}}$ and the MPC query action $\mathbf{a}_{\text{MPC}}$ are recorded.

During each environment step, transitions are stored in both buffers:

$$(\mathbf{o}, \mathbf{s}, \mathbf{a}_{\text{MPC}}, \mathbf{a}_{\text{NN}}, \mathbf{o}', \mathbf{s}', r, d, \mathbf{g}, \mathbf{y}_{\text{aux}}) \rightarrow \{\mathcal{B}_{\text{expert}}, \mathcal{B}_{\text{dagger}}\}, \quad (2)$$

where $\mathbf{y}_{\text{aux}}$ denotes auxiliary task labels (relative pose, velocity, and angular velocity).

During training, we sample mini-batches with a hybrid strategy:

$$\mathcal{B}_{\text{batch}} = \text{Sample}(\mathcal{B}_{\text{expert}}, N_e) \cup \text{Sample}(\mathcal{B}_{\text{dagger}}^{\text{old}}, N_o) \cup \text{Sample}(\mathcal{B}_{\text{dagger}}^{\text{recent}}, N_r), \quad (3)$$

where recent samples are drawn from the tail of the DAgger buffer to prioritize recent experience. This sampling strategy ensures diversity while emphasizing recent policy behavior.

B. Policy Network Architecture

The vision-based policy π_θ takes a sequence of visual observations $[\mathbf{o}_{t-H+1}, \dots, \mathbf{o}_t]$ as input, where H denotes the history length. The network adopts a three-module architecture:

1) *Perception Module*: Extracts spatial features from individual frames. We implement two variants:

- *ResNet-based encoder*: Comprises residual blocks with batch normalization and adaptive average pooling, outputting feature vectors of dimension d_1 .
- *Vision Transformer (ViT)*: Splits images into patches, applies linear projection, and processes through transformer encoder layers with multi-head self-attention.

The perception module outputs frame-wise features $\mathbf{f}_t \in \mathbb{R}^{d_1}$ and provides auxiliary predictions of relative pose $\hat{\mathbf{y}}_t^{\text{pose}} = [\hat{\mathbf{p}}_r^\top, \hat{\mathbf{R}}_r^\top]^\top \in \mathbb{R}^9$ (3D position + 6D rotation) through a dedicated auxiliary head.

2) *Temporal Module*: Aggregates temporal information from the feature sequence. We support three architectures:

- *Gated Recurrent Unit (GRU)*: Processes the sequence recurrently, outputting the final hidden state $\mathbf{h}_T \in \mathbb{R}^{d_2}$.
- *Temporal Transformer*: Appends a learnable summary token and applies self-attention over the temporal dimension.
- *Temporal Convolutional Network (TCN)*: Employs dilated causal convolutions with residual connections.

This module provides auxiliary predictions of relative velocity and angular velocity $\hat{\mathbf{y}}_t^{\text{dyn}} = [\hat{\mathbf{v}}_r^\top, \hat{\boldsymbol{\omega}}_r^\top]^\top \in \mathbb{R}^6$.

3) *Control Module*: A multi-layer perceptron (MLP) concatenates the temporal feature vector with the goal position $\mathbf{g} \in \mathbb{R}^3$ and outputs the mean $\boldsymbol{\mu}$ and log-standard deviation $\log \boldsymbol{\sigma}$ of a Gaussian policy:

$$\boldsymbol{\mu}, \log \boldsymbol{\sigma} = \text{MLP}([\mathbf{h}_T; \mathbf{g}]). \quad (4)$$

Actions are sampled using the reparameterization trick with tanh squashing for bounded action spaces.

C. Dynamic Loss Weighting for IL-RL Integration

The key innovation of our approach is the dynamic integration of imitation learning and reinforcement learning through a unified loss function with time-varying weights. Unlike sequential pre-training and fine-tuning, our method seamlessly blends both objectives throughout training.

1) *Imitation Learning Loss*: The IL objective minimizes the mean squared error between the policy output and expert (MPC) actions. For each sampled transition, we construct the IL loss using the expert action label:

$$\mathcal{L}_{\text{IL}}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{o}, \mathbf{g}, \mathbf{a}_{\text{MPC}}) \sim \mathcal{B}_{\text{batch}}} [\|\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{o}, \mathbf{g}) - \mathbf{a}_{\text{MPC}}\|^2], \quad (5)$$

where the expectation is taken over all samples from both expert and DAgger buffers, as both contain expert action labels.

2) *Reinforcement Learning Loss*: The RL objective follows the Soft Actor-Critic (SAC) framework [?] with entropy regularization. The policy loss is:

$$\mathcal{L}_{\text{RL}}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{B}_{\text{dagger}}} [\alpha \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t | \mathbf{s}_t) - Q_{\phi}(\mathbf{s}_t, \mathbf{a}_t)], \quad (6)$$

where $\alpha > 0$ is the temperature parameter, and the expectation is taken only from DAgger buffer samples (i.e., states visited by the policy). The log-probability accounts for the tanh squashing function:

$$\log \pi_{\boldsymbol{\theta}}(\mathbf{a} | \mathbf{s}) = \log \mathcal{N}(\boldsymbol{\epsilon}; \mathbf{0}, \mathbf{I}) - \sum_{i=1}^d \log(1 - \tanh^2(z_i)) - \sum_{i=1}^d \log \frac{\text{high}_i - \text{low}_i}{2}, \quad (7)$$

where $\mathbf{z} = \boldsymbol{\mu}_{\boldsymbol{\theta}} + \boldsymbol{\sigma}_{\boldsymbol{\theta}} \odot \boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

3) *Asymmetric Critic*: We employ an asymmetric actor-critic architecture [?] where the critic receives privileged state information while the actor uses only visual observations. The twin Q-networks Q_{ϕ_1}, Q_{ϕ_2} minimize the Bellman error:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}') \sim \mathcal{B}_{\text{batch}}} [(Q_{\phi}(\mathbf{s}, \mathbf{a}) - y)^2], \quad (8)$$

with the target value:

$$y = r + \gamma(1-d) \left(\min_{i=1,2} Q_{\bar{\phi}_i}(\mathbf{s}', \mathbf{a}') - \alpha \log \pi_{\boldsymbol{\theta}}(\mathbf{a}' | \mathbf{s}') \right), \quad (9)$$

where $\mathbf{a}' \sim \pi_{\boldsymbol{\theta}}(\cdot | \mathbf{s}')$ and $Q_{\bar{\phi}}$ denotes the target network updated via soft interpolation:

$$\bar{\phi} \leftarrow \tau \phi + (1 - \tau) \bar{\phi}. \quad (10)$$

Notably, the critic is trained on all transitions (both expert and DAgger), using the actually executed action ($\mathbf{a}_{\text{MPC}}$ for expert data, $\mathbf{a}_{\text{NN}}$ for DAgger data):

$$\mathbf{a}_{\text{critic}} = \begin{cases} \mathbf{a}_{\text{MPC}} & \text{if } s \in \mathcal{B}_{\text{expert}} \\ \mathbf{a}_{\text{NN}} & \text{if } s \in \mathcal{B}_{\text{dagger}} \end{cases} \quad (11)$$

4) *Dynamic Loss Weighting Schedule*: The combined policy loss integrates IL, RL, and auxiliary objectives with dynamic weighting:

$$\mathcal{L}_{\text{policy}} = w_{\text{RL}}(k) \cdot \mathcal{L}_{\text{RL}} + (1 - w_{\text{RL}}(k)) \cdot w_{\text{IL}} \cdot \mathcal{L}_{\text{IL}} + w_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}, \quad (12)$$

where k denotes the update step, w_{IL} is a fixed scaling factor for the IL loss, and the RL weight $w_{\text{RL}}(k)$ follows a linear schedule:

$$w_{\text{RL}}(k) = \min \left(1, \frac{k}{K_{\text{window}}} \right) \cdot w_{\text{RL}}^{\text{target}}. \quad (13)$$

Here, K_{window} is the transition window size controlling the speed of IL-to-RL transition, and $w_{\text{RL}}^{\text{target}}$ is the final RL weight (typically < 1 to maintain some IL guidance). This annealing strategy ensures that early training benefits from strong expert guidance while gradually allowing the policy to explore and improve beyond the expert through RL.

5) *Auxiliary Task Learning*: To enhance representation learning, we incorporate auxiliary tasks predicting physical quantities from visual observations:

$$\mathcal{L}_{\text{aux}} = w_p \mathcal{L}_{\text{pos}} + w_r \mathcal{L}_{\text{rot}} + w_v \mathcal{L}_{\text{vel}} + w_{\omega} \mathcal{L}_{\text{ang}}, \quad (14)$$

where each component is computed as:

$$\mathcal{L}_{\text{pos}} = \frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{p}}_r^{(t)} - \mathbf{p}_r^{(t)}\|^2, \quad (15)$$

$$\mathcal{L}_{\text{rot}} = \frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{R}}_r^{(t)} - \mathbf{R}_r^{(t)}\|_F^2, \quad (16)$$

$$\mathcal{L}_{\text{vel}} = \frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{v}}_r^{(t)} - \mathbf{v}_r^{(t)}\|^2, \quad (17)$$

$$\mathcal{L}_{\text{ang}} = \frac{1}{T} \sum_{t=1}^T \|\hat{\boldsymbol{\omega}}_r^{(t)} - \boldsymbol{\omega}_r^{(t)}\|^2. \quad (18)$$

For rotation representation, we employ the 6D continuous representation [?] to avoid discontinuities associated with Euler angles or quaternions.

D. Reward Design

The reward function at time t comprises multiple components:

$$r_t = r_{\text{prog}} + r_{\text{align}} + r_{\text{vel}} + r_{\text{act}} + r_{\text{body}} + r_{\text{time}} + r_{\text{terminal}}, \quad (19)$$

where:

- $r_{\text{prog}} = w_1 \cdot \Delta d_{\text{gate}}$: Progress reward based on distance reduction to the next gate.
- $r_{\text{align}} = w_2 \cdot (\mathbf{v} \cdot \mathbf{d}_{\text{target}} / \|\mathbf{d}_{\text{target}}\|)$: Velocity alignment with target direction.
- $r_{\text{vel}} = w_3 \cdot \mathbf{v} \cdot \mathbf{d}_{\text{target}}$: Velocity projection toward target.
- $r_{\text{act}} = -w_4 \cdot \|\mathbf{a}\|^2$: Control magnitude penalty.
- $r_{\text{body}} = -w_5 \cdot \|\boldsymbol{\omega}\|^2$: Body rate penalty to reduce motion blur.
- $r_{\text{time}} = -w_6$: Constant time cost to encourage faster flight.
- r_{terminal} : Success bonus or crash penalty at episode termination.

E. Training Algorithm

The complete training procedure is summarized in Algorithm 1. Unlike conventional approaches that separate IL pre-training from RL fine-tuning, our method simultaneously collects expert and on-policy data, dynamically adjusting the IL-RL loss balance throughout training.

F. Implementation Details

Learning Rate Warm-up: To stabilize early training, we employ a linear learning rate warm-up:

$$\eta_k = \eta_{\text{base}} \cdot \min\left(1, \frac{k}{K_{\text{warm}}}\right). \quad (20)$$

Gradient Clipping: We apply gradient clipping with maximum norm c_{max} to prevent exploding gradients:

$$\mathbf{g} \leftarrow \frac{\mathbf{g}}{\max(1, \|\mathbf{g}\|_2 / c_{\text{max}})}. \quad (21)$$

Weight Initialization: We employ task-specific strategies: Kaiming initialization for convolutional layers, orthogonal initialization for recurrent weights, and Xavier uniform for transformer projections. The policy output layers use bounded uniform initialization to ensure reasonable initial action variance.

Automatic Entropy Tuning: The temperature parameter α can be automatically adjusted to achieve a target entropy $\mathcal{H}_{\text{target}} = -\dim(\mathcal{A})$, eliminating the need for manual tuning.

Algorithm 1 Simultaneous IL-RL Training with Dynamic Loss Weighting

Require: Expert policy π_{expert} , initial parameters θ, ϕ , loss weights, transition window K_{window}

- 1: Initialize buffers $\mathcal{B}_{\text{expert}}, \mathcal{B}_{\text{dagger}}$, target networks $\bar{\phi} \leftarrow \phi$
- 2: **for** episode = 1, 2, ... **do**
- 3: Reset environment, $\mathbf{h}_0 \leftarrow \mathbf{0}$
- 4: **if** episode mod expert_freq = 0 **then**
- 5: Execute π_{expert} , store $(\mathbf{o}, \mathbf{s}, \mathbf{a}_{\text{MPC}}, \mathbf{a}_{\text{NN}}, \dots)$ in both buffers
- 6: **else**
- 7: Execute π_{θ} , query π_{expert} for $\mathbf{a}_{\text{MPC}}$, store in both buffers
- 8: **end if**
- 9: **for** each environment step **do**
- 10: **if** total_steps mod update_interval = 0 **then**
- 11: **for** $i = 1$ to updates_per_episode **do**
- 12: Sample batch via Eq. 3
- 13: Compute dynamic weight $w_{\text{RL}}(k)$ via Eq. 13
- 14: Compute critic action: $\mathbf{a}_{\text{critic}} = \mathbb{I}_{\text{expert}} \cdot \mathbf{a}_{\text{MPC}} + (1 - \mathbb{I}_{\text{expert}}) \cdot \mathbf{a}_{\text{NN}}$
- 15: Update critic via Eq. 8–9
- 16: Update policy via Eq. 12 (IL on all data, RL on DAgger only)
- 17: Update temperature α
- 18: Soft update targets via Eq. 10, $k \leftarrow k + 1$
- 19: **end for**
- 20: **end if**
- 21: **end for**
- 22: **end for**
- 23: **return** Optimized policy θ^*

REFERENCES